

Санкт-Петербургский Национальный Исследовательский Университет
Информационных Технологий, Механики и Оптики

Факультет Инфокоммуникационных Технологий

Практическая работа №7-8
«Разработка проекта на Django»

Выполнил:

Гайдаренко Никита Кириллович

Группа:

K3321

Проверила:

Марченко Елена Вадимовна

Санкт-Петербург,

2025

Цель:

Создать проект на Django. Проект должен содержать 3 страницы: Главная, о разработке, страница обратной связи. У всех страниц должен быть одинаковый хедер и футтер. С использованием инструментов Django создать страницу обратной связи.

Задачи:

- Изучить основные компоненты Django;
- Создать базовый Django-проект с тремя страницами;
- Создать общий хедером и футером для всех страниц
- Реализовать функционал обратной связи.

Ход работы:

Был установлен Django при помощи команды:

```
pip install django
```

Далее командой:

```
python3 -m django startproject main.
```

создал проект. Автоматически создана папка myproject и внутри нее файлы: `__init__.py`, `asgi.py`, `settings.py`, `urls.py` и `wsgi.py`, а также файл `manage.py`.

Для реализации функционала было создано отдельное приложение при помощи команды:

```
python3 manage.py startapp main
```

Оно содержит в себе файлы: `__init__.py`, `admin.py`, `apps.py`, `forms.py`, `models.py`, `tests.py`, `urls.py`, `views.py`.

Описание задач каждого файла приведена ниже:

1. `main/templates/main/base.html`
 - a. Это базовый шаблон, который определяет общую структуру всех страниц сайта
 - b. Содержит общие элементы: `header`, `footer`, меню
 - c. Включает основные CSS стили

- d. Определяет блоки, которые могут быть переопределены в дочерних шаблонах (например, {% block content %})
2. main/views.py
 - a. Содержит классы представлений (views), которые обрабатывают запросы пользователей
 - b. Определяет логику отображения страниц
 - c. Включает классы HomeView, AboutView, ContactView для соответствующих страниц
 - d. Обрабатывает формы (форму обратной связи)
 3. main/urls.py
 - a. Определяет URL-маршруты приложения
 - b. Сопоставляет URL-адреса с соответствующими представлениями
 - c. Содержит пути для главной страницы, страницы "О разработке" и формы обратной связи
 4. main/models.py
 - a. Определяет структуру базы данных
 - b. Содержит модели данных (ContactMessage для хранения сообщений обратной связи)
 - c. Определяет поля и их типы для каждой модели
 5. main/forms.py
 - a. Содержит определения форм для обработки пользовательского ввода
 - b. Определяет поля форм, их стили и валидацию
 - c. В данном случае содержит форму обратной связи
 6. main/admin.py
 - a. Настраивает административный интерфейс Django
 - b. Определяет, как модели будут отображаться в админ-панели
 - c. Настраивает фильтры и поиск для моделей
 7. myproject/settings.py
 - a. Основной файл настроек проекта
 - b. Содержит настройки базы данных, установленных приложений, middleware
 - c. Определяет настройки безопасности, локализации и статических файлов
 8. myproject/urls.py
 - a. Корневой файл URL-маршрутов проекта

- b. Включает URL-маршруты из приложений
 - c. Определяет путь к административной панели
- 9. `manage.py`
 - a. Утилита командной строки для управления проектом Django
 - b. Позволяет запускать сервер разработки, создавать миграции, выполнять тесты и т.д.
- 10. `myproject/wsgi.py` и `myproject/asgi.py`
 - a. Файлы для развертывания приложения на сервере
 - b. WSGI (Web Server Gateway Interface) - для традиционных веб-серверов
 - c. ASGI (Asynchronous Server Gateway Interface) - для асинхронных серверов
- 11. `main/apps.py`
 - a. Конфигурация приложения
 - b. Определяет метаданные приложения
 - c. Настраивает автоматическое поле для моделей
- 12. `main/templates/main/home.html`, `about.html`, `contact.html`
 - a. Шаблоны для конкретных страниц
 - b. Наследуют структуру из `base.html`
 - c. Определяют уникальный контент для каждой страницы

Проект имеет структуру представленную на рисунке 1.

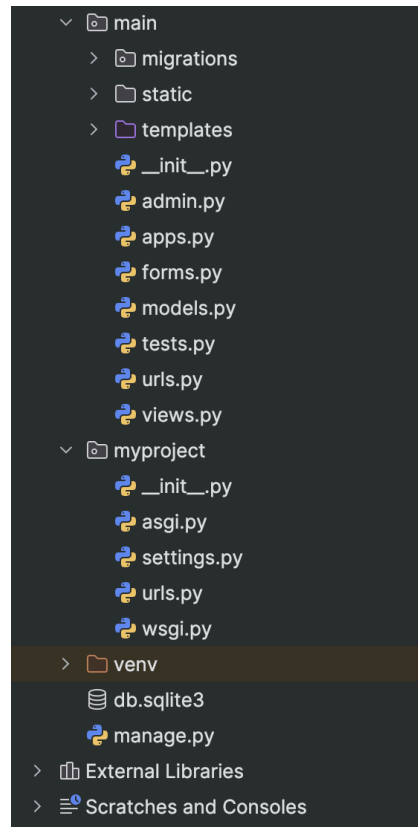


Рисунок 1 - Структура проекта

Далее в файле `models.py`, определяем структуру данных (модели), которые Django автоматически преобразует в таблицы базы данных. Каждый класс модели соответствует таблице в базе данных. Эта модель будет хранить данные обратной связи (рисунок 2). Каждое поле — это столбец в таблице базы данных.

```
from django.db import models

class ContactMessage(models.Model): 5 usages
    name = models.CharField(max_length=100, verbose_name='Имя')
    email = models.EmailField(verbose_name='Email')
    message = models.TextField(verbose_name='Сообщение')
    created_at = models.DateTimeField(auto_now_add=True, verbose_name='Дата создания')

    class Meta:
        verbose_name = 'Сообщение'
        verbose_name_plural = 'Сообщения'
        ordering = ['-created_at']

    def __str__(self):
        return f'{self.name} - {self.email}'
```

Рисунок 2 - Код файла `models.py`

После была создана Django форма, которая служит для безопасного сбора и обработки пользовательских данных. В данном проекте форма обратной связи (рисунок 3) реализована через `ModelForm` — специальный класс, который автоматически создает форму на основе модели.

```
class ContactForm(forms.ModelForm): 2 usages
    class Meta:
        model = ContactMessage
        fields = ['name', 'email', 'message']
        widgets = {
            'name': forms.TextInput(attrs={'class': 'form-control'}),
            'email': forms.EmailInput(attrs={'class': 'form-control'}),
            'message': forms.Textarea(attrs={'class': 'form-control', 'rows': 5}),
        }
        labels = {
            'name': 'Ваше имя',
            'email': 'Ваш email',
            'message': 'Ваше сообщение',
        }
    }
```

Рисунок 3 - Код файла `forms.py`

В данном случае, `ModelForm` автоматически генерирует форму с полями, соответствующими модели.

`Meta` - это внутренний класс для настроек: о `model = Feedback` указывает, какая модель используется. о `fields` - список полей модели, которые должны быть в форме.

Далее реализован файл `views.py` (рисунок 4), который содержит функции, которые обрабатывают запросы и возвращают ответы: отображение главной страницы, формы обратной связи и т.д.

```

class HomeView(TemplateView): 2 usages
    template_name = 'main/home.html'

    /about/
class AboutView(TemplateView): 2 usages
    template_name = 'main/about.html'

    /contact/
class ContactView(FormView): 2 usages
    template_name = 'main/contact.html'
    form_class = ContactForm
    success_url = '/contact/'

    def form_valid(self, form):
        form.save()
        return super().form_valid(form)

    def get_context_data(self, **kwargs):
        context = super().get_context_data(**kwargs)
        context['form'] = self.form_class()
        return context

```

Рисунок 4 - Код файла views.py

Также был настроен файл urls.py необходимый для маршрутизации конкретного приложения, которые подключаются к главным маршрутам проекта. На рисунке 5 представлен результат настройки.

```

urlpatterns = [
    /
    path('', HomeView.as_view(), name='home'),
    /about/
    path('about/', AboutView.as_view(), name='about'),
    /contact/
    path('contact/', ContactView.as_view(), name='contact'),
]

```

Рисунок 5 - Код файла urls.py

Далее происходила написание визуальной части приложения

Папка `templates/` папка, которая используется для хранения HTML шаблонов.

Файл `base.html` – это базовый шаблон сайта, который используется как основа для других HTML-шаблонов. Согласно заданию, содержит общие хидеры и футеры. (Рисунок 6)

Файл `home.html` - это «Главная» страница сайта. (Рисунок 6)

Файл `about.html` - это страница «О разработке», которая также имеет подключение CSS-стиля для отображения анимации голубей (Рисунок 7)

Файл `contact.html` - это страница «Обратная связь» (Рисунок 8)

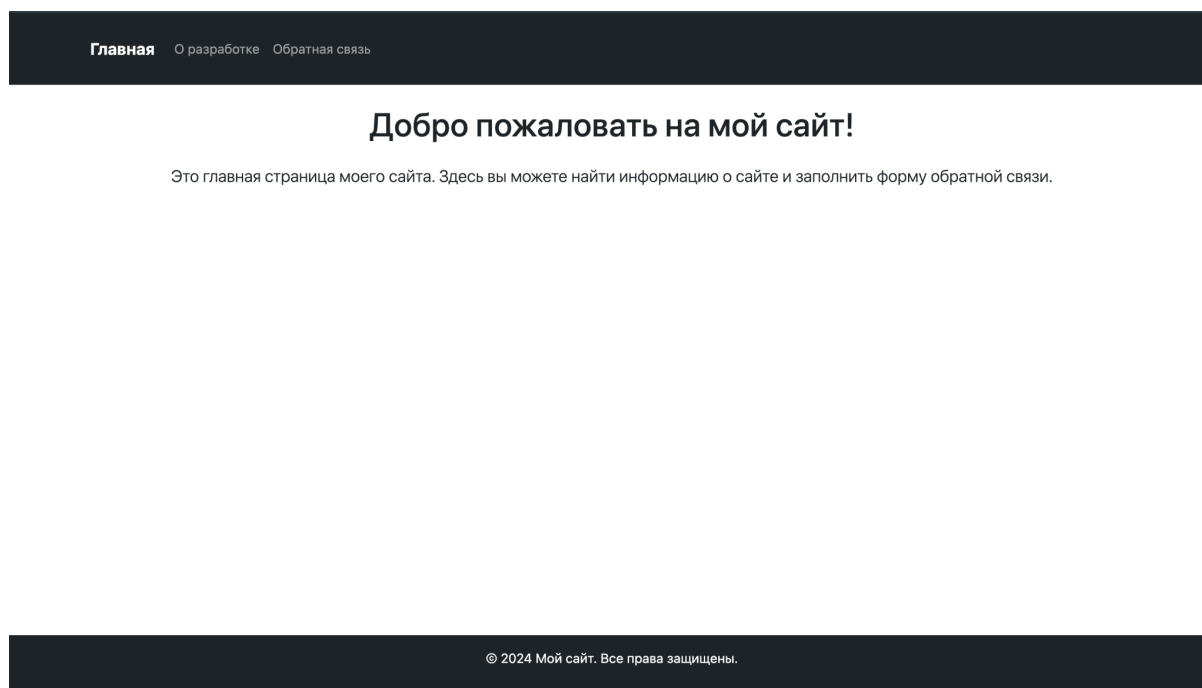


Рисунок 6 - Главная страница

О разработке

Технологии

Используемые технологий:

- Django - фреймворк. Сайт с использованием Spring Boot был бы получше
- Bootstrap
- HTML

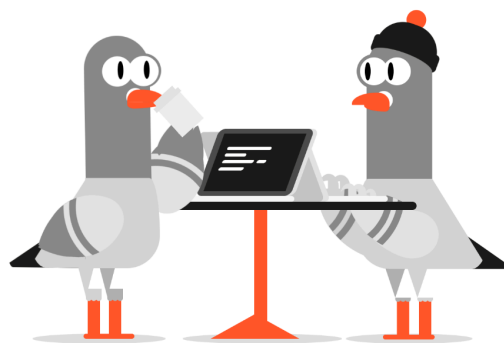


Рисунок 7 - Страница «О разработке»

Обратная связь

Ваше имя:

Nikita

Ваш email:

Tablicysostup@gmail.com

Ваше сообщение:

Крутой сайт!!!!

Отправить

Рисунок 8 - Страница «Обратная связь»

Вывод:

В результате выполнения лабораторной работы были получены практические навыки и умения по созданию веб-приложений на фреймворке Django: был разработан проект с нуля, реализована структура с несколькими страницами, выполнена настройка маршрутов и шаблонов, создана форма обратной связи с обработкой данных, а также применены базовые возможности шаблонов для подключения общих компонентов — хидеров и футеров. Работа способствовала углублению понимания архитектуры Django и принципов организации веб-приложений.